# Real-Time UPI Payment Failure Diagnosis, Reconciliation and Fraud Risk Monitoring System

*A simulated UPI payment platform with two added features: concurrent fraud-risk detection and real-time failure diagnosis*

Project Report — Problem, Solution, Design and Plan

Computer Science and Engineering, KLE Technological University

**In one line: a normal UPI-style app for sending money, with two engines underneath. Six concurrent fraud checks decide whether a payment is suspicious. A rule-based diagnosis engine explains why a stuck payment is stuck, what to do next, and roughly when it will resolve.**

# 1. The Problem

When a person sends money by UPI, two things must happen: the sender's account is debited and the receiver's account is credited. Sometimes only the first happens. The sender has lost the money, the receiver has not got it, and the sender does not know why, or when it will be fixed. A second, separate problem is fraud: suspicious transactions that must be flagged before they complete.

## Who is affected

- **Senders** see a vague "pending" message and often retry, which risks a duplicate debit.
- **Receivers** (shops, individuals) do not know whether to hand over goods or service.
- **Operations teams** must search logs across systems to understand each stuck case.

# 2. Why the Problem Happens

A UPI payment is two separate database transactions in two separate banks, connected through a switch (NPCI). Each bank guarantees correctness only inside its own database. Debit and credit cannot be one all-or-nothing step, so there is always a window in which:

- the request to the receiver's bank is waiting in a queue
- a network message was lost and has not been retried
- the receiver's bank is slow, overloaded or down
- the response was sent but got lost on the way back
- the credit happened but the confirmation never arrived

In each case the sender's side genuinely cannot know what happened on the other side. This is the **distributed transaction problem**. Under a network partition (CAP theorem) a system cannot be fully available and fully consistent at the same moment, so real UPI apps put the payment into a pending state and resolve it afterwards.

# 3. What Existing UPI Already Does, and the Gap

| Existing mechanism | How it works | Limitation |
|---|---|---|
| Status-check API | Sender's bank asks for the current state of a transaction ID. | Returns a state, not an explanation or a time estimate. |
| Batch reconciliation | Banks compare settlement records at fixed intervals and fix mismatches. | Not instant; runs after the fact. |
| Auto-reversal | If no credit within a fixed window, money returns to the sender. | User waits with no information meanwhile. |

**The gap: the time between a payment getting stuck and the batch job running. In that window the user sees only "pending". We do not claim UPI has no solution. We improve the real-time explanation layer on top of it.**

## 4. What Recent Research Already Solves

| | Paper 1: Fraud | Paper 2: Failure diagnosis |
|---|---|---|
| Title | Fraud Detection in UPI Payments Using Tabular Machine Learning Models | A Feature Engineering Approach for Business Impact-Oriented Failure Detection in Distributed Instant Payment Systems |
| Source | Chaudhary, Singh, Singh, Zaidi, Jain. IJRASET, Vol. 13, Oct 2025. DOI 10.22214/ijraset.2025.74922 | L. Porcelli, Bank of Italy. arXiv:2510.21710, revised Aug 2025 |
| Problem | UPI fraud grows, no public UPI dataset exists, data is categorical and heavily imbalanced. | In a real instant-payment system no single participant sees the whole transaction, so faults between banks are missed by component monitoring. |
| Method | Synthetic dataset of 100,000 transactions (0.63% fraud). CatBoost compared with XGBoost. | Three timing features from message timestamps, an anomaly detector, and a simple rule-based explainer that separates internal from external failure. |
| Why that method | CatBoost handles categorical features natively and copes with class imbalance. Deep learning judged unnecessary for tabular data. | Deep models are hard to explain and slow for real-time use. Hand-built features show where the fault is by design. |
| Result | CatBoost ROC-AUC 0.870 vs 0.844; recall 0.60 vs 0.42. | Correctly localised a real Aug 2023 incident and four stress scenarios. |
| What it leaves open | Only fraud scoring. No payment-state tracking, reconciliation or recovery estimate. | Infrastructure-level monitoring, not a user-facing explanation, duplicate protection or recovery time. |

Together the two papers support our split: use machine learning where the question is statistical (fraud), and use simple explainable rules where the question is deterministic and must be instant (diagnosis).

## 5. Our Solution

A simulated UPI platform (send money, track a payment, payment history) with two engines underneath. For most transactions the app behaves like any normal UPI app. The two engines add the value:

| | Feature 1: Fraud risk | Feature 2: Failure diagnosis |
|---|---|---|
| Question answered | Is this transaction suspicious? | Why is this payment stuck, what happens next, and when will it resolve? |
| Method | 6 concurrent checks (thread pool) plus a trained model | Payment state machine plus rule-based decision table |
| Needs history? | Yes (fraud patterns) | No for diagnosis; simulator history for the time estimate |
| Output | Approve / Review / Block with a reason per check | State, likely reason, next action, estimated recovery time, alerts |

Supporting mechanisms: idempotency keys to block duplicate payments, an append-only event log, role-specific alerts for sender, receiver and operations, and an operations dashboard.

## Current versus proposed flow

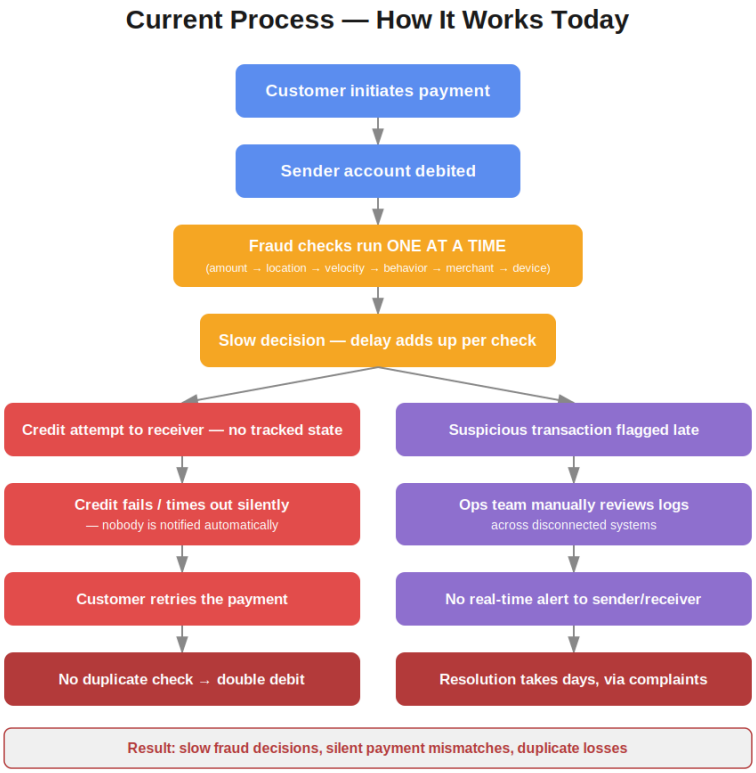



*Figure 1. How a stuck payment is handled today.*

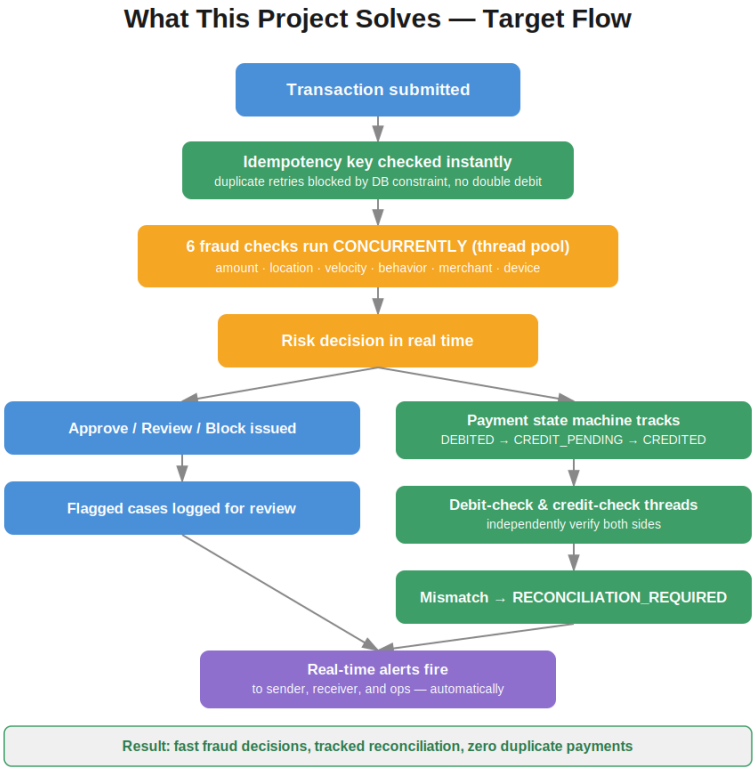



*Figure 2. The flow this project builds.*

# 6. Feature 1 in Detail: Concurrent Fraud-Risk Detection

A shared ThreadPoolExecutor runs six independent checks at the same time for each transaction. Each returns a score from 0 to 100 and a reason. The checks only read data, so they need no locks.

| Check | What it looks at | Weight |
|---|---|---|
| Amount | Amount compared with the customer's usual amount | 0.25 |
| Location | New location or impossible travel | 0.20 |
| Velocity | Number of transactions in the last N minutes | 0.20 |
| Behavior | Deviation from the customer's profile | 0.15 |
| Merchant | Risk lookup for the merchant | 0.10 |
| Device / IP | New or unrecognised device | 0.10 |

The weighted sum maps to a decision: **0–30 Approve, 31–70 Review, 71–100 Block**. Weights are a starting point and are tuned during testing.

### Machine-learning layer

- **Baseline:** Logistic Regression. **Final model:** CatBoost, chosen for native categorical handling and robustness to imbalance, following Paper 1.
- **Metrics:** recall, precision, F1, confusion matrix and ROC-AUC. Plain accuracy is misleading because fraud is rare.
- **Concurrency proof:** log thread name and start/end time per check, then benchmark sequential time (sum of latencies) against concurrent time (about the slowest single check) on the same batch.

Threads are used because the checks are I/O-bound (lookups and simulated service calls), not CPU-bound, so threads rather than processes are the right choice.

## 7. Feature 2 in Detail: Failure Diagnosis and Reconciliation

### Payment state machine

**INITIATED, PROCESSING, DEBITED, CREDIT_PENDING, CREDITED, COMPLETED**, with failure states **TIMEOUT, FAILED, RECONCILIATION_REQUIRED**. Every transition goes through one function that rejects illegal moves and writes an immutable row to an append-only event log.

| From | Allowed next states |
|---|---|
| INITIATED | PROCESSING, FAILED |
| PROCESSING | DEBITED, FAILED, TIMEOUT |
| DEBITED | CREDIT_PENDING |
| CREDIT_PENDING | CREDITED, TIMEOUT, RECONCILIATION_REQUIRED |
| CREDITED | COMPLETED |
| TIMEOUT | CREDITED (status check finds it), RECONCILIATION_REQUIRED, FAILED |
| RECONCILIATION_REQUIRED | COMPLETED (resolved), FAILED (reversed to sender) |

### Diagnosis decision table (rule-based, no training data)

The engine reads real-time signals: debit response, credit response, error code, timeout, retry count and duplicate key. It outputs the likely reason, the next state and the action.

| Signals | Likely reason | State | Action and user message |
|---|---|---|---|
| Debit OK, credit OK, ack received | Normal success | COMPLETED | Confirm to user. |
| Debit OK, credit error code | Receiver-side failure | RECONCILIATION_REQUIRED | Start reversal workflow; tell sender not to retry; alert ops. |
| Debit OK, credit timeout | Receiver slow or down | TIMEOUT, then RECONCILIATION_REQUIRED if status check still shows no credit | No duplicate debit; run status check; alert sender and receiver. |
| Credit applied, ack lost | Lost confirmation | COMPLETED after status check | Verify via status check; update both sides. |
| Debit failed | Sender bank failure | FAILED | Tell user nothing was deducted. |
| Invalid receiver | Bad account details | FAILED (before debit) | Ask user to correct details. |
| Insufficient balance | Balance check failed | FAILED (before debit) | Show balance error. |
| Same idempotency key again | Duplicate or retry | Unchanged | Return the existing transaction; no new debit. |
| Network failure | Connectivity | PENDING / TIMEOUT | Retry status check with backoff. |
| Bank or server unavailable | Bank outage | TIMEOUT | Longer recovery estimate; alert. |

## Debit-check and credit-check

After a debit succeeds, independent checks verify each side. If a debit exists but no matching credit appears within the timeout, the transaction becomes RECONCILIATION_REQUIRED and alerts go out: the sender is told not to retry, the receiver is told the payment is pending, and operations is notified.

## Recovery-time estimate

For stuck cases the system groups past simulator runs by failure type and reports the typical resolution time for that type, starting with simple median-per-type and moving to a regression only if needed. The result is always shown as an **estimate, never a guarantee**, and is kept separate from rule-verified states.

## Duplicate payment prevention

Every payment carries an idempotency key stored with a **database unique constraint**. A repeated key returns the existing transaction. A plain check-then-insert is not enough because two simultaneous requests can both pass the check; the database constraint is what makes it safe.

## 8. Where Machine Learning Is and Is Not Used

| Part of the system | Approach | Why |
|---|---|---|
| Payment state and failure diagnosis | Rules | Needs no history, must be instant, must be explainable. |
| Duplicate detection, reconciliation trigger | Rules and database constraints | Deterministic logic is exact. |
| Fraud scoring | Weighted rules plus CatBoost | A statistical pattern problem that benefits from historical examples. |
| Recovery-time estimate | Statistics on simulator history | Depends on past behaviour of each failure type. |

## 9. Dataset Strategy

No real UPI transaction data is publicly available, so we use two sources, as both research papers did:

- **Simulator-generated data:** our own transaction stream (normal, delayed, stuck, duplicate) for state tracking, diagnosis testing and the recovery-time estimate.
- **Synthetic fraud dataset:** a labelled, imbalanced, categorical dataset (fraud around 0.5–1%) with fields such as amount, time, device, location, receiver novelty and transaction frequency, used to train and evaluate the fraud model.

We state plainly that this proves the concept and is not production validation. No real bank or NPCI data is used.

## 10. Architecture and Technology

| Layer | Component | Technology |
|---|---|---|
| Interface | UPI app (send, track, history) and operations dashboard | Web UI; Streamlit for the dashboard |
| API | REST endpoints and payment flow | Python, FastAPI |
| Core | State machine, idempotency, bank simulators | Python, SQLAlchemy |
| Intelligence | Fraud checks, fraud model, diagnosis rules, recovery estimate | ThreadPoolExecutor, scikit-learn / CatBoost |
| Data | Transactions, events, risk results, alerts, cases | SQLite (PostgreSQL later) |
| Alerts | Dashboard alerts, then email | smtplib |

### Database tables

customers, accounts, merchants, devices, transactions (idempotency_key unique), transaction_events (append-only), risk_results, alerts, fraud_cases.

### Main endpoints

POST /transactions, GET /transactions/{id}, GET /payment-status/{id}, GET /risk/{id}, GET /alerts, GET /cases, GET /logs/{id}, POST /sim/config (switch bank simulator mode for demos).

## 11. Live Demonstration Scenario

- **Step 1.** User sends Rs. 1,000 from the UPI app.

- **Step 2.** Fraud checks run concurrently and approve it; the thread log shows them overlapping.
- **Step 3.** The sender is debited; the receiver-bank simulator is switched to TIMEOUT.
- **Step 4.** The system detects the timeout, moves the payment to TIMEOUT and runs a status check.
- **Step 5.** No credit found, so the state becomes RECONCILIATION_REQUIRED.
- **Step 6.** The user sees a real-time alert: do not retry, with an estimated recovery time.
- **Step 7.** The operations dashboard lists the case with its diagnosed reason.
- **Step 8.** The same payment is retried with the same key; the system returns the existing transaction and does not debit again.

## 12. Evaluation

| Area | Metrics | Most important because |
|---|---|---|
| Fraud detection | Recall, precision, F1, confusion matrix, ROC-AUC | Fraud is rare, so accuracy hides missed fraud; recall matters most. |
| Failure diagnosis | Correct state identified, correct reason, false-diagnosis rate, reconciliation success | Wrong guidance to a user is the costly error. |
| Concurrency | Sequential time vs concurrent time on the same batch | Direct proof that threading helps. |
| Real-time behaviour | Detection latency, response time, throughput | Shows the system reacts fast enough. |
| Duplicate prevention | Same key sent concurrently produces exactly one transaction | Core safety property. |

## 13. Team Plan

| Role | Owns | Main deliverables |
|---|---|---|
| Member A: Payment core | Backend | Schema, APIs, state machine, idempotency, bank simulators |
| Member B: Fraud and diagnosis | fraud-intel | 6 threaded checks and benchmark, fraud model, diagnosis table, recovery estimate |
| Member C: UI and operations | frontend | UPI app, ops dashboard, alerts, end-to-end test scenarios |
| Guide | Review | Phase sign-offs, pull-request review, demo and viva readiness |

Phases: 0 design lock, 1 skeleton, 2 happy path end to end, 3 the two features, 4 integration hardening with 10 test scenarios, 5 polish and demo.

## 14. Limitations and What We Do Not Claim

- We simulate sender bank, receiver bank and gateway. This is not a real UPI, NPCI or bank implementation.
- A system cannot know a bank's internal reason for a failure unless that information is exposed. Our reasons are inferred from observable signals.
- Fraud data is synthetic, so reported scores demonstrate the method, not real-world accuracy.
- Recovery times are estimates from simulated behaviour, not promises of refund or settlement.
- Production security and compliance (for example PCI-DSS) are out of scope.

## 15. Future Scope

- Learn recovery times with a regression model as more history accumulates.
- Use real anonymised data if a partner bank or institution can provide it.
- Streaming ingestion (Kafka) and PostgreSQL for scale.
- SMS or WhatsApp alerts and a dispute-handling workflow.
- Graph-based fraud-ring detection.

## 16. Quick Answers for the Viva

| Question | Short answer |
|---|---|
| What is the problem? | Money is debited but not credited and the user cannot tell why or when it resolves. |
| Why does it happen? | Two banks, two databases, an unreliable network; debit and credit cannot be one atomic step. |
| What does UPI already do? | Status check, batch reconciliation, auto-reversal. We add the real-time explanation. |
| How do you find the reason? | A rule-based decision table over real-time signals; no historical data needed. |
| Where is ML used? | Fraud scoring and the recovery-time estimate only. |
| What do the six threads do? | They score fraud risk concurrently. They do not diagnose the fault; the rules do. |
| Which dataset? | Synthetic fraud data plus our simulator's logs; limitation stated openly. |
| How do you stop duplicates? | A database unique constraint on the idempotency key. |
| What happens on debit without credit? | TIMEOUT, status check, RECONCILIATION_REQUIRED, alerts, reversal workflow. |

## References

[1] R. Chaudhary, S. Singh, R. Singh, H. Zaidi, K. Jain, "Fraud Detection in UPI Payments Using Tabular Machine Learning Models," IJRASET, Vol. 13, Oct 2025, doi:10.22214/ijraset.2025.74922.
[2] L. Porcelli, "A Feature Engineering Approach for Business Impact-Oriented Failure Detection in Distributed Instant Payment Systems," arXiv:2510.21710, revised Aug 2025.